

DSA0502 – Query Processing for Data Science	
Course Faculty	Dr. B .SHYAMALA GOWRI
Reg. No.	192324164
Student Name	Sweetie Roselin A
Date	30/07/2026

CO2 AT3 – DEBUGGING TEST

Q1. Debug the SQLite Program

Given Program

```
import sqlite3
conn = sqlite3.connect(college.db)
cursor = conn.cursor
cursor.execute("""
CREATE TABLE Student(
id INTEGER PRIMARY KEY
name TEXT,
marks INTEGER
)
""")
cursor.execute("INSERT INTO Student VALUES(1*,'John',90)")
conn.close()
```

Errors Identified

Error	Reason
college.db is not enclosed in quotes	Database name must be a string.
conn.cursor is missing ()	cursor() creates the cursor object.
Missing comma after PRIMARY KEY	Invalid SQL syntax.
INSERT statement contains 1*	Invalid SQL value.
conn.commit() is missing	Changes are not saved permanently.

Cursor is not closed	Resources should be released after use.
CREATE TABLE may fail if the table already exists	Use IF NOT EXISTS.

Correct Program

```
import sqlite3

conn = sqlite3.connect("college.db")

cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS Student(
    id INTEGER PRIMARY KEY,
    name TEXT,
    marks INTEGER
)
""")

cursor.execute("INSERT INTO Student VALUES(1,'John',90)")

conn.commit()

cursor.close()

conn.close()
```

Purpose of the Corrections

- Enclosed "college.db" in quotes because the database name must be a string.
- Added cursor() to create the cursor object.
- Inserted the missing comma after PRIMARY KEY.
- Used IF NOT EXISTS to avoid errors when the table already exists.
- Added commit() to permanently save the inserted record.
- Closed the cursor to release database resources.

Q2. Debug the CRUD Program

Given Code

```
cursor.execute("SELECT * FROM Student")

for row in cursor.fetchone():

    print(row)
```

Errors Identified

1. fetchone() retrieves only one record.
2. The loop prints individual column values instead of complete rows.
3. UPDATE Student SET marks=95 updates every record.
4. DELETE FROM Student removes every record.
5. commit() is missing.
6. Cursor is not closed.

Correct Program

```
import sqlite3

conn = sqlite3.connect("college.db")

cursor = conn.cursor()

cursor.execute("SELECT * FROM Student")

rows = cursor.fetchall()

for row in rows:

    print(row)

cursor.execute("UPDATE Student SET marks=95 WHERE id=1")

cursor.execute("DELETE FROM Student WHERE id=2")

conn.commit()

cursor.close()

conn.close()
```

Why Every Record Was Modified or Deleted

Without a WHERE clause:

UPDATE Student SET marks=95;
updates every record.

Similarly,

DELETE FROM Student;
deletes every record in the table.

Q3. Debug the MySQL Connection Program

Given Code

```
connection=mysql.connector.connect(host="localhost",user="root"password="root",database="college")
```

Errors Identified

- Missing comma after "root".
- Poor code formatting.
- Cursor and connection are not closed.

Correct Program

```
import mysql.connector

connection = mysql.connector.connect(
    host="localhost",
    user="root",
    password="root",
    database="college"
)

cursor = connection.cursor()

cursor.execute("SELECT * FROM student")

for row in cursor:
    print(row)

cursor.close()

connection.close()
```

Why the Original Program Failed

The missing comma causes a:

SyntaxError: invalid syntax

Python cannot parse the connection statement.

Q4. Debug the PostgreSQL Program

Given Code

```
cursor.execute(  
    "INSERT INTO Student VALUES(%s,%s,%s)",  
    (1,"Ram")  
)
```

Errors Identified

- Three placeholders are provided.
- Only two values are supplied.
- marks value is missing.
- Cursor is not closed.

Correct Program

```
import psycopg2  
conn = psycopg2.connect(  
    dbname="college",  
    user="postgres",  
    password="admin"  
)  
cursor = conn.cursor()  
cursor.execute(  
    "INSERT INTO Student VALUES(%s,%s,%s)",  
    (1, "Ram", 90)  
)  
conn.commit()  
cursor.execute("SELECT * FROM Student")  
print(cursor.fetchone())  
cursor.close()  
conn.close()
```

Effect of Parameter Mismatch

A parameter mismatch raises a:

ProgrammingError

because SQL expects three values but receives only two.

Q5. Multiple SQL Errors

Wrong Statement	Correct Statement	SQL Category
CREATE TABLE Student(id INTEGER,name TEXT marks INTEGER)	CREATE TABLE Student(id INTEGER, name TEXT, marks INTEGER);	DDL
INSERT Student VALUES(...)	INSERT INTO Student VALUES(1,'Anu',89);	DML
SELECT name marks FROM Student;	SELECT name, marks FROM Student;	DQL
UPDATE Student marks=90 WHERE id=1;	UPDATE Student SET marks=90 WHERE id=1;	DML
DELETE Student WHERE id=2;	DELETE FROM Student WHERE id=2;	DML

Q6. Debug the Transaction Program

Errors Identified

- rollback() is immediately followed by commit().
- After rollback, there are no changes left to commit.
- Inserting duplicate IDs may violate the primary key constraint.

Correct Program

```
import sqlite3
conn = sqlite3.connect("college.db")
cursor = conn.cursor()
try:
    cursor.execute("UPDATE Student SET marks=80 WHERE id=2")
    conn.commit()
```

except Exception:

```
    conn.rollback()
```

finally:

```
    cursor.close()
```

```
    conn.close()
```

Difference Between commit() and rollback()

- commit() permanently saves all changes.
- rollback() cancels all uncommitted changes and restores the previous state.

Q7. Debug the Parameterized Query

Given Code

```
id = 2
```

```
cursor.execute("SELECT * FROM Student WHERE id=" + id)
```

Errors Identified

- String and integer cannot be concatenated.
- Vulnerable to SQL Injection attacks.

Correct Program

```
id = 2
```

```
cursor.execute(  
    "SELECT * FROM Student WHERE id=?",  
    (id,) )  
print(cursor.fetchall())
```

Why Is It Unsafe?

If user input is:

1 OR 1=1

the query may return all records.

Parameterized queries treat user input as data, preventing SQL Injection.

Q8. Database Fetch Program

Errors Identified

- rows[20] may raise an IndexError if fewer than 21 records exist.
- Executing SQL after closing the cursor causes a ProgrammingError.

Correct Program

```
cursor.execute("SELECT * FROM Student")
rows = cursor.fetchall()
if len(rows) > 20:
    print(rows[20])
else:
    print("Less than 21 records.")
cursor.execute("SELECT * FROM Student")
print(cursor.fetchall())
cursor.close()
```

Q9. Integrated Debugging

Errors Identified

1. Table may already exist.
2. Duplicate primary key.
3. Incorrect column name (mark instead of marks).
4. Incorrect table name (Students instead of Student).
5. Missing commit().
6. Cursor is not closed.

Correct Program

```
import sqlite3
conn = sqlite3.connect("college.db")
cursor = conn.cursor()
cursor.execute("""
CREATE TABLE IF NOT EXISTS Student(
    id INTEGER PRIMARY KEY,
```



```

        name TEXT,
        marks INTEGER
    )
    """
cursor.execute("INSERT OR REPLACE INTO Student VALUES(1,'Asha',90)")
cursor.execute("UPDATE Student SET marks=95 WHERE id=1")
cursor.execute("SELECT name, marks FROM Student")
print(cursor.fetchall())
conn.commit()
cursor.close()
conn.close()

```

Possible Exceptions

Error	Exception
Duplicate primary key	IntegrityError
Invalid column name	OperationalError
Invalid table name	OperationalError
Table already exists	OperationalError

Q10. Complete Database Application Debugging

Errors Identified

1. Missing commit().
2. fetchone() displays only one record.
3. UPDATE statement has no WHERE clause.
4. Incorrect delete syntax (==).
5. Cursor is not closed.
6. Database changes are not committed.

Correct Program

```

import sqlite3

conn = sqlite3.connect("college.db")

cursor = conn.cursor()

```

```

# Insert
cursor.execute("INSERT INTO Student VALUES(3,'Kiran',90)")

# Read
cursor.execute("SELECT * FROM Student")
rows = cursor.fetchall()

for row in rows:
    print(row)

# Update
cursor.execute(
    "UPDATE Student SET marks=95 WHERE id=3"
)

# Delete
cursor.execute(
    "DELETE FROM Student WHERE id=3"
)

conn.commit()
cursor.close()
conn.close()

```

Benefits of the Corrections

- commit() permanently saves all database changes.
- fetchall() retrieves and displays every record.
- WHERE ensures only the intended record is updated.
- DELETE FROM ... WHERE removes only the specified record.
- Closing the cursor releases database resources.
- Proper transaction handling maintains database consistency.

Predicted Output

Initial Student Table

ID	Name	Marks
1	John	90
2	Anu	89

Output After Insert Operation

(1, 'John', 90)

(2, 'Anu', 89)

(3, 'Kiran', 90)

After updating Kiran's marks to **95** and deleting the record with **ID = 3**, the final table becomes:

(1, 'John', 90)

(2, 'Anu', 89)